Configuration Management

CSE 362

Software Development and Professional Practice

Winter 2023

© Royal Military College of Canada

Introduction

- Over the lifetime of a large software system, many artifacts are to be managed:
  - **Source code** components
  - **Documentation**
  - **Change requests**.
  - Etc.

- A plan is needed to:
  - Manage the changes to the artifacts
  - Track the status of the artifacts
  - Provide a historical record of the artifacts.

➔ That is where we need the process of configuration management

What is Configuration Management

- The process of <u>controlling the evolution</u> of a software system.

- How?

- Establish mechanisms to
  - Simplify code sharing and other documents
  - Be able to revert to an older version ("undo")
  - Coherently integrate contributions from team members ("merge")
  - Notify interested parties about new modifications ("reporting")
  - Track software issues (e.g., bugs)
  - Create an auditing trail ("archiving")

Why Configuration Management?

- Configuration Management or SCM (Software Configuration Management) should allow us to answer the following questions:
  - When was this software module last changed?
  - Who made changes in this line of code?
  - What is the difference between the current version and last week's?
  - How many lines of code did we change in this release?
  - Which files are changed most frequently?

Definitions

- A **software configuration** is a collection of work products ("artifacts") created during a project lifetime
- A **Software configuration item** (**SCI**) is any piece of knowledge (a.k.a. "work product" or "artifact") created by a person (developer or customer)
- A snapshot of a software configuration or a configuration item represents the **version** of that item at a specific time
- A **commit** (or "check-in") refers to submitting a software configuration to the project database ("repository")
- A **build** is a version of a program code (part of a software configuration), and a process by which program code is converted into a stand-alone form that can be run on a computer ("executable code")

How to implement configuration management

- Step 1: Identification
  - The first action towards SCM is <u>information gathering</u>
  - Configuration data should be aggregated and compiled from different application environments.
  - Any secret data like passwords and keys should be identified and securely encrypted and stored.
  - At this point configuration data should be organized into data files

How to implement configuration management

- Step 2: Baseline
  - After configuration data has been aggregated and organized a baseline can be established.
  - A baseline configuration is a known state of configuration that will successfully operate the dependent software without error.

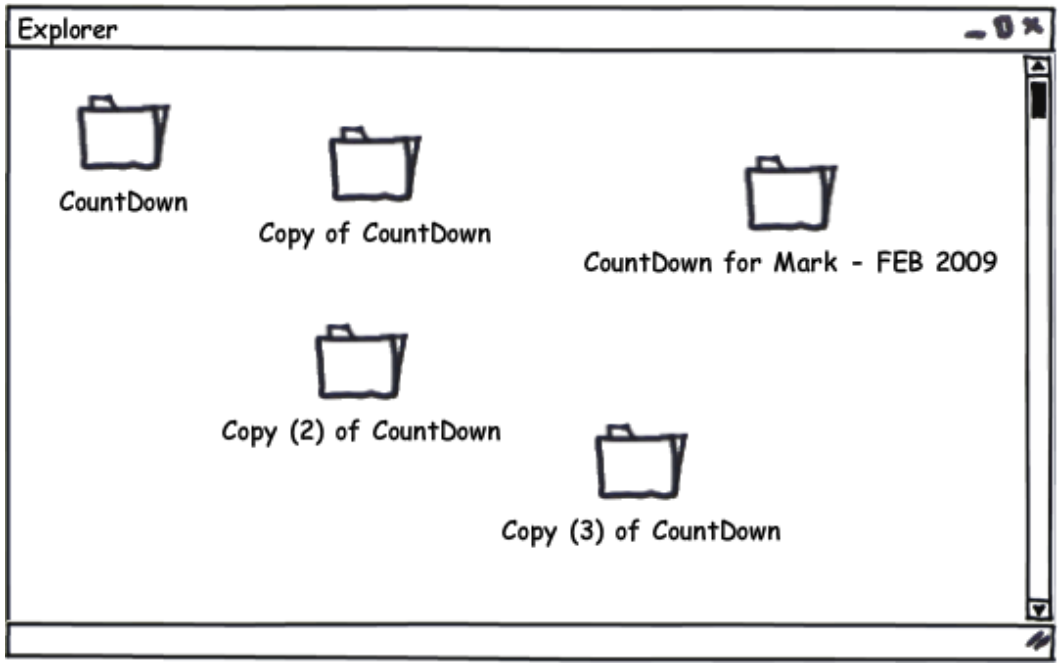
How to implement configuration management

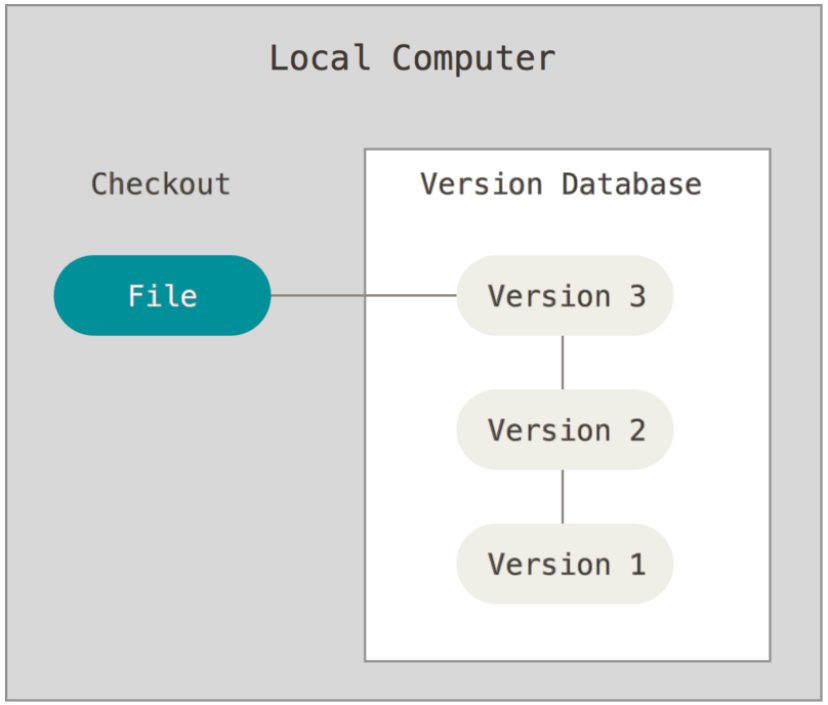
- Step 3: Version Control
  - Your development project should use a version control system.
  - Git is the leading tool for version control.
  - More on that in a bit.

How to implement configuration management

- Step 4: Auditing
  - Having configuration data organized and added to a repository enables collaboration and visibility into the system's configuration.
  - The popular pull request workflow that software teams use to review and edit code can then be applied to configuration data files.
  - This helps build add accountability and visibility.

Version Control: What is it?





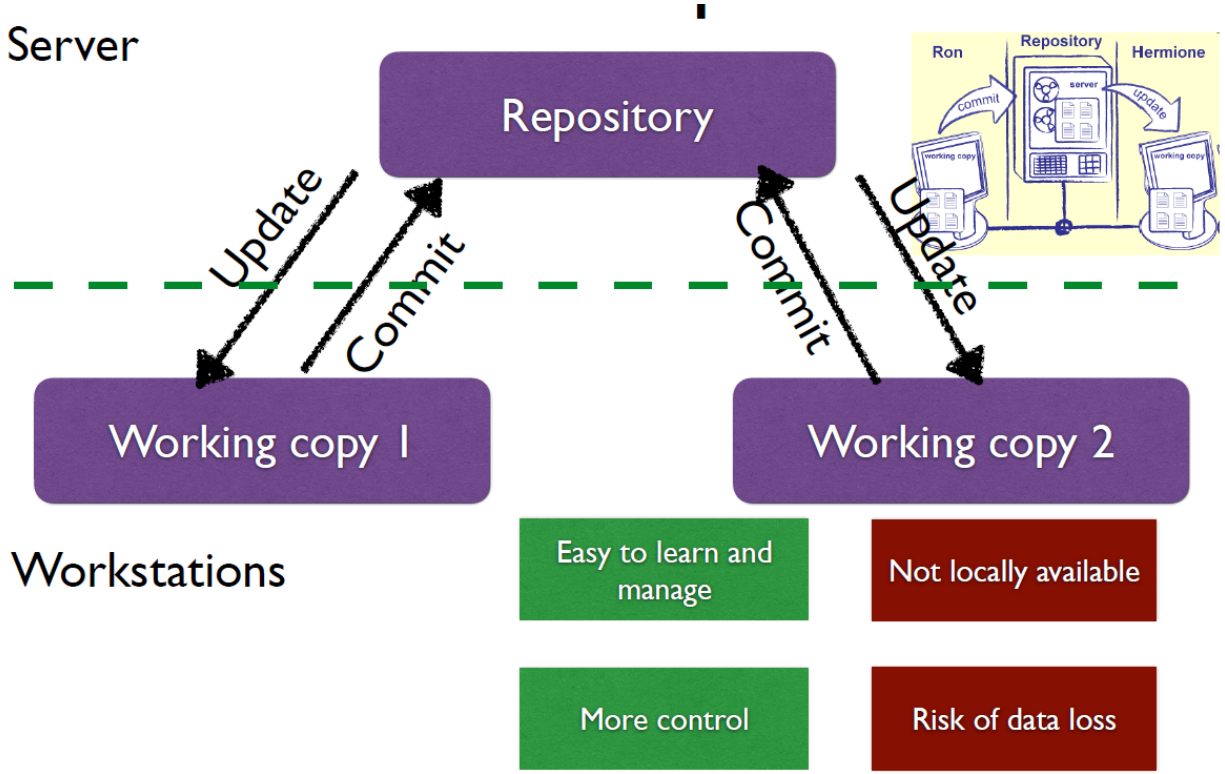
- A method for recalling versions of a codebase
- Keeping a record of changes
- Who did what and when in the system
- Save yourself when things inevitably go wrong

Version Control: 2 Types

- Centralised VCS
- Distributed VCS

Centralized VCS: What older VCS used

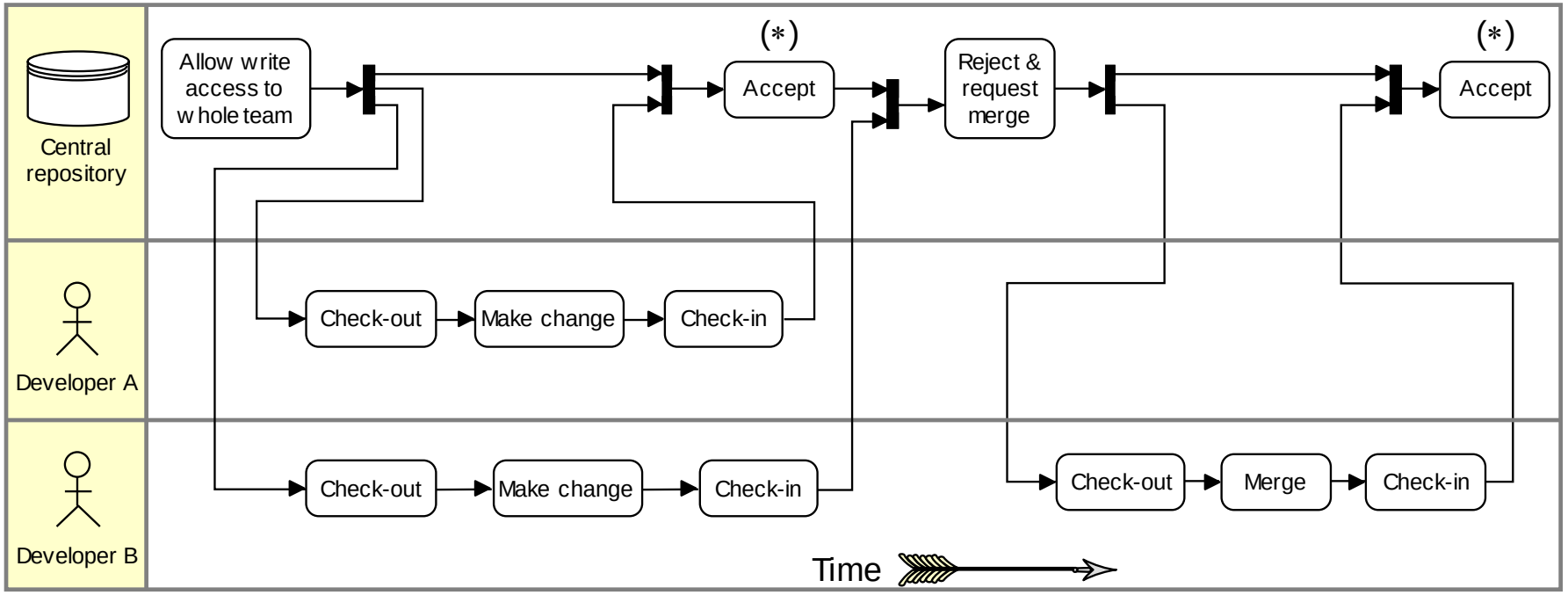
- A **single** authoritative data source (repository)
- Check-outs and check-ins are done with reference to this central repository



Centralized VCS

- What if two or more people want to edit the same file at the same time?
- Option 1: make them take turns
  - But then only one person can be working at any time
  - And how do you enforce the rule?
- Option 2: patch up differences afterwards
  - Requires a lot of re-working – Stuff always gets lost

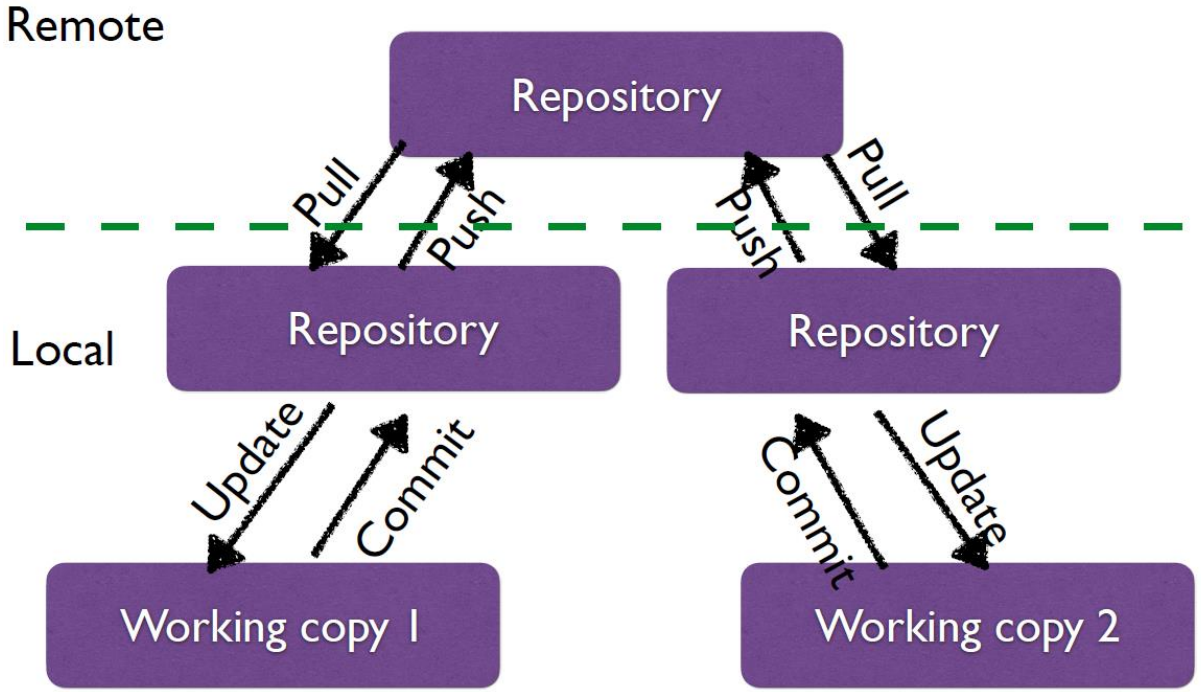
Centralized VCS: A scenario



(*) Assuming no other commits in the meantime; otherwise need to merge

Distributed VCS – what Git uses

- Each user gets his or her own repository *and* working copy.
- After you commit, others have no access to your changes until you push your changes to the central repository.
- For others to see your changes, 4 things must happen: What are they?

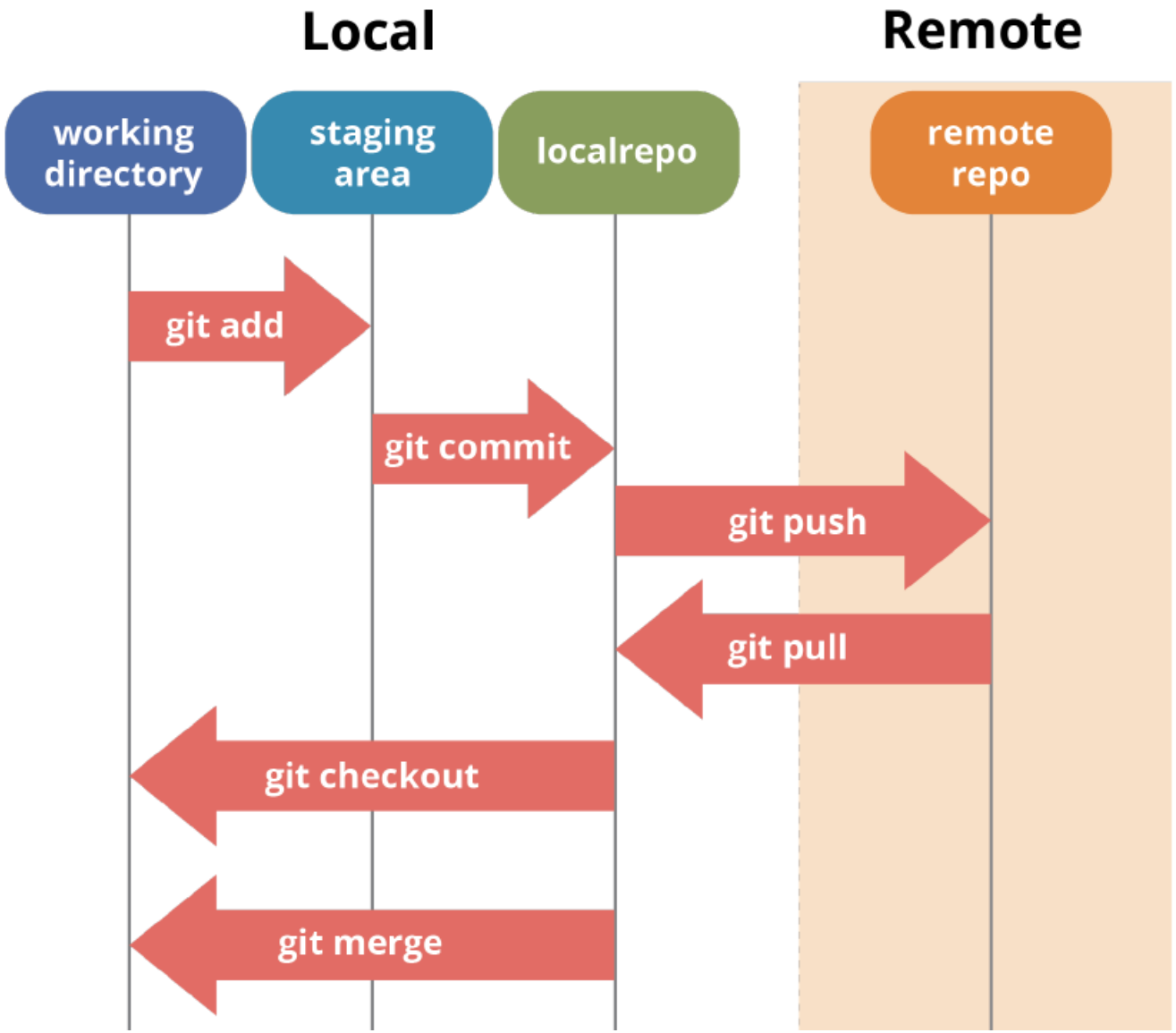


Git

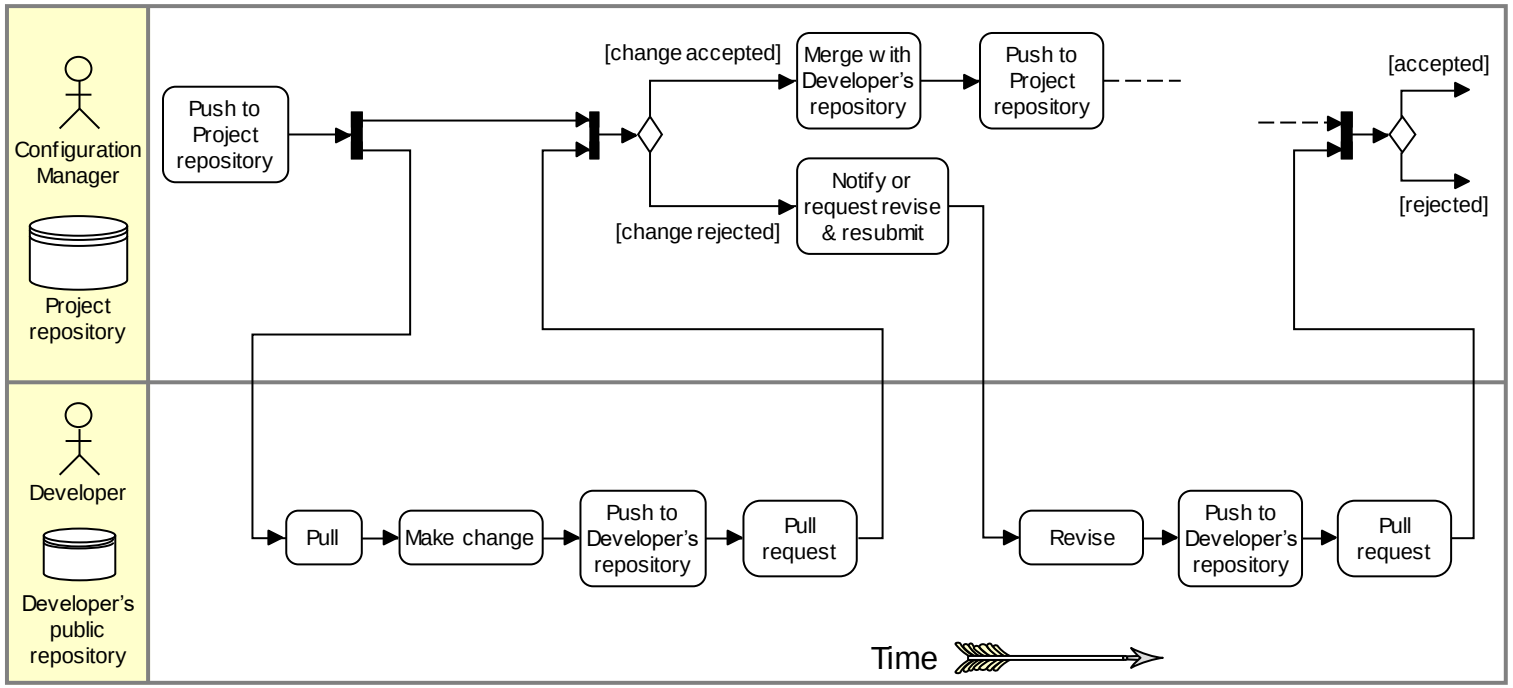
- Git is a foundational tool in higher-level configuration management.
- Git is the industry-leading <u>version control system</u> to track code changes.
- Git is an open-source version control system that was started by Linus Torvalds, the principal force behind the Linux kernel.

- Adding configuration management data alongside code in a Git repository provides a holistic version control view of an entire project.

Git Structure



Distributed VCS: A scenario



1. The configuration manager pushes the current version to the central project repository
2. A contributor clones that repository and makes changes
3. The contributor pushes the changed version to his own public repository
4. The contributor notifies the configuration manager requesting to pull changes
5. The configuration manager adds the contributor's repository as a remote and merges locally
6. The configuration manager pushes merged changes to the central project repository

Git Terms

- You clone to create your repository.
- You add to put files in the staging area.
- When you are ready, you commit files to your local repository. You have created a snapshot of your repository.
- A push sends your snapshot out to the remote repository. Other users will get your updates by using a "fetch" or a "pull":

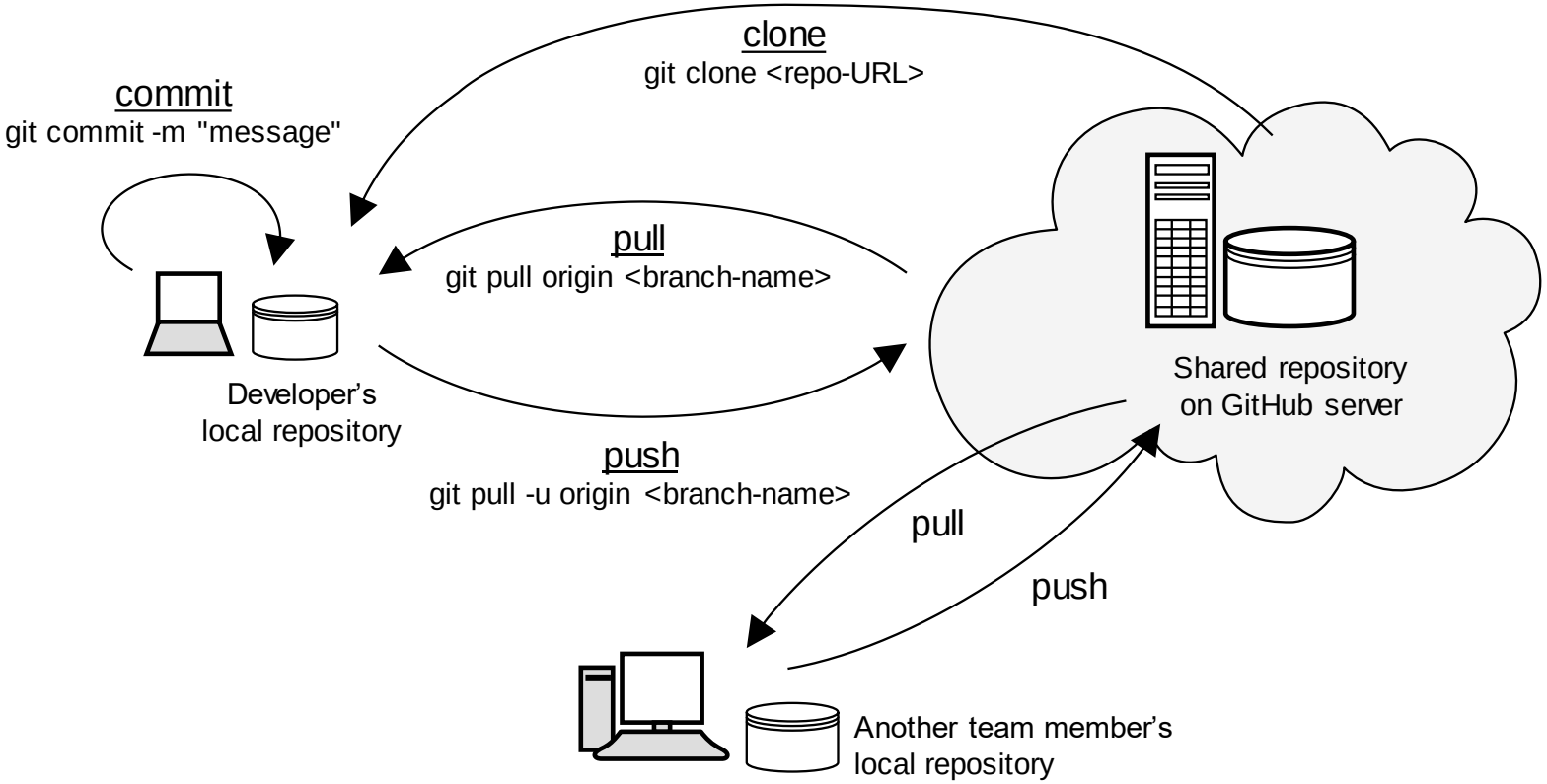
Git Terms, Cont.

- A fetch just updates your local repository and adds information to your log files and version records. It is pretty harmless and should be done frequently.

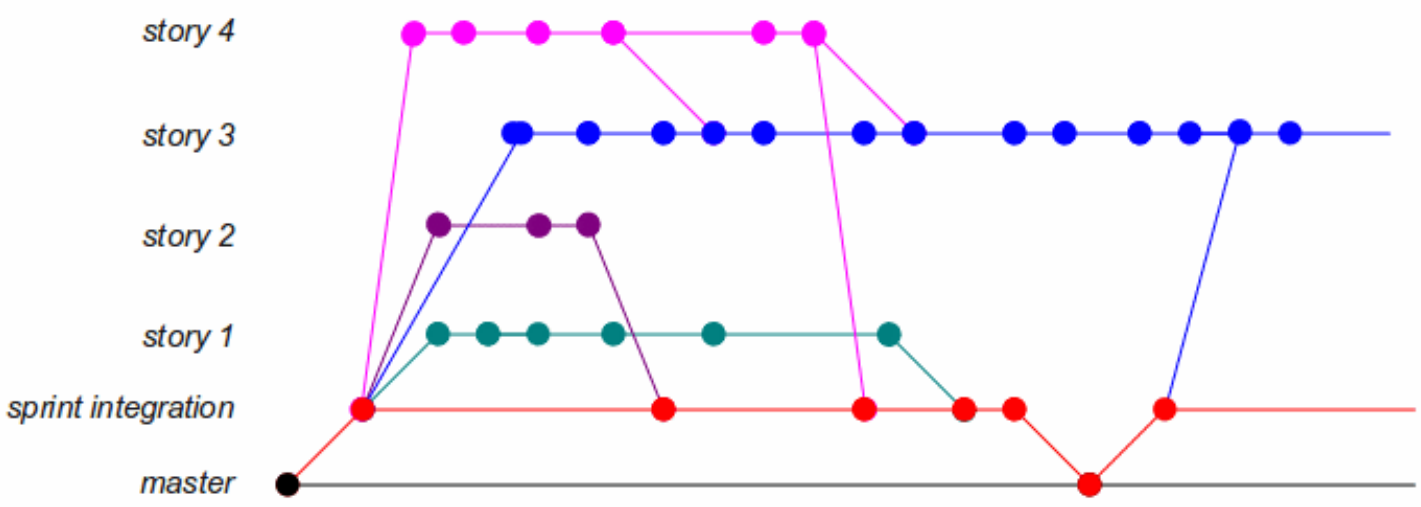
- A pull carries out a fetch followed by a merge of the Head branch. This will update the files in your working directory to the latest versions.

- If two people have changed the same file before the merge operation you get a merge conflict that has to be resolved.

Few Git Commands



Git Branching

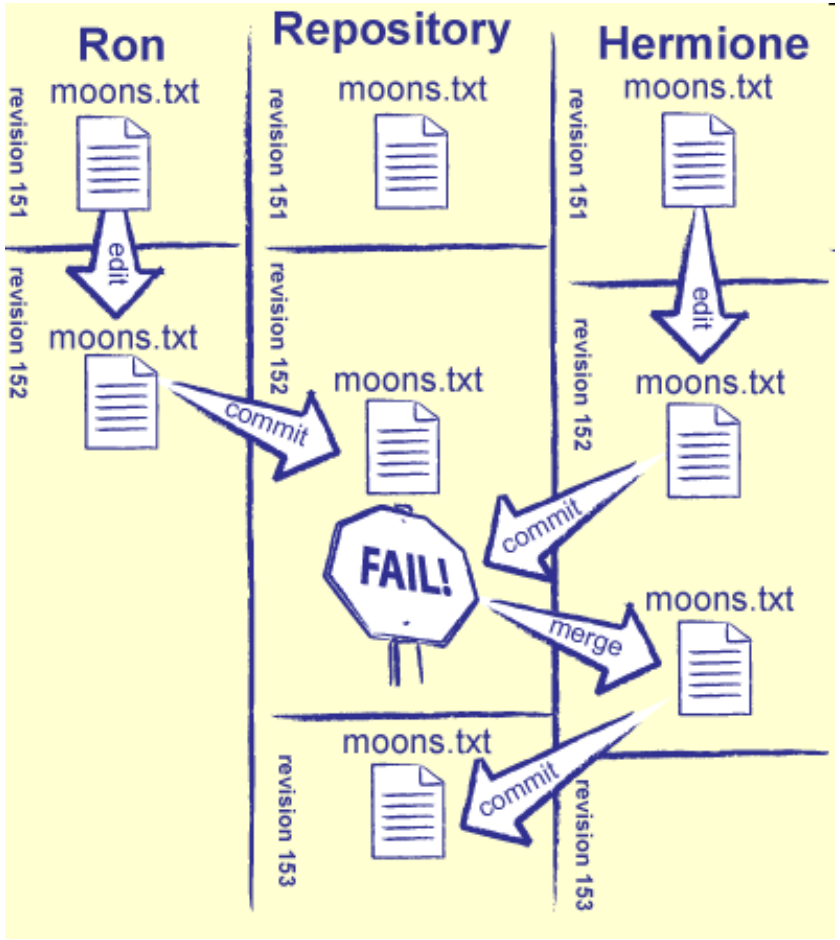


- Features can be developed in an isolated way by creating a branch. Eventually (at the end of a sprint?) it will need to merge back to the "master" branch.

Resolving Conflicts

Using Git "terminology":

- Hermione attempts a <u>push</u>, but Ron has done a <u>push</u> after Hermione has <u>fetched</u> (or <u>cloned</u>).
- Hermione's push will be rejected, even if she has not changed the same files that Ron has changed.
- Hermione must do a <u>fetch</u> and <u>merge</u> to get Ron's changes before she can push.
- Ron's changes will be applied and then Hermione's changes will be re-applied.



Resolving Conflicts

- It may be that changes do not overlap, so a merge will work automatically, but if there is a conflict:
- The resolution process allows one person to view a merged file, with the differences marked. This person decides which revisions to keep/discard.

How to actually use Git

"SO, WHERE DID YOU TWO MEET?"
AT THE OFFICE
AT STARBUCKS
GITHUB
stickycomics.com
WINDOWS USERS
MAC USERS
LINUX USERS

Next…

People Management